

INDEX

Name ADITYA CHITGOKER Std 7 Sec F

Roll No. Subject OS School/College

School/College Tel. No. Parents Tel. No.

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
i	2/3	2 nd smallest / largest element	}	Rm 3/3/24
ii	2/3	Sum of left diagonal of matrix		
iii	2/3	Sum of rows & columns of matrix		
iv	2/3	Duplicate Elements in Array		
v	2/3	Delete element at position		
1 (a)	2/3	Turnaround time &	}	Rm 3/3/24
(b)		Wait Time for FCS FCS & SJF		
(c)	10/3	Priority pre-emptive & non-pre-emptive.		Rm 10/3/24
(d)	17/3	Round-Robin.		Rm 17/3/24
2	24/3	Multi-Level Queue Scheduling.		Rm 24/3/24
3	31/3	RMS & EDF		Rm 31/3/24
4	21/3	Producer Consumer Dining Philosopher	} (8)	Rm 28/4/24
5	5/5	Banker's Algorithm Deadlock Detection.	} (10)	Rm
6	12/5	Fit Algorithm	→ (10)	Rm

[illegible]

① Program to find 2nd smallest / largest element of an array.

```
#include <stdio.h>
int main () {
    int a[] = {5, 4, 3, 2, 1};
    int n = sizeof(a) / sizeof(n);
    for (int i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (a[j] > a[j+1]) {
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    printf ("2nd largest := %d", a[n-2]);
    printf ("2nd smallest := %d", a[1]);
    return 0;
}
```

I (a)

Program to find Turnaround Time & Waiting Time for the FCFS & SJF

Date: / /
Page: /

FCFS:

#include <stdio.h>

struct processes {

int id, AT, BT, CT, TAT, WAT, RT, Q;

int main () {

int n;

printf ("Number of processes: "); scanf ("%d", &n);

struct processes p[n];

for (int i=0; i<n; i++) {

p[i].id = i+1;

printf ("Arrival Time of P%d: ", p[i].id);

scanf ("%d", &p[i].AT);

printf ("Burst Time of P%d: ", p[i].id);

scanf ("%d", &p[i].BT);

}

for (int i=0; i<n; i++) {

for (j=0; j<n-i-1; j++) {

if (p[j].AT > p[j+1].AT) {

struct processes temp = p[j];

p[j] = p[j+1];

p[j+1] = temp;

}

int time passed = 0, sumTAT=0, sumWAT=0, sumRT=0;

for (int i=0; i<n; i++) {

if (time passed < p[i].AT) time passed = p[i].AT;

p[i].CT = time passed + p[i].BT;

p[i].TAT = p[i].CT - p[i].AT;

}

p[i].WAT = p[i].TAT - p[i].BT;

time passed += p[i].BT

sumTAT += p[i].TAT

sumWAT += p[i].WAT

sumRT += p[i].RT

printf ("Average TAT: %.2f\n", (float)sumTAT/n);

printf ("Average WAT: %.2f\n", (float)sumWAT/n);

printf ("Average RT: %.2f\n", (float)sumRT/n);

return 0;

}

Output:

Number of Processes: 2

Arrival time of P1: 0

Burst Time of P1: 3

Arrival Time of P2: 1

Burst Time of P2: 2

Average TAT: 3.50

Average WAT: 1.00

Average RT: 1.00

Date: / /
Page: /

(b)

```

#include <stdio.h>

struct Process {
    int id, at, bt, et, tot, wt, rt;
};

int main() {
    int n;
    printf("Enter no. of processes: "); scanf("%d", &n);

    struct Process p[n];
    for(int i=0; i<n; i++) {
        p[i].id = i+1;
        printf("Arrival Time of p[%d]: ", i);
        scanf("%d", &p[i].at);
        printf("Burst Time of p[%d]: ", i);
        scanf("%d", &p[i].bt);
    }

    int completed = 0, time = 0, sumAT = 0, sumWT = 0;
    int isComplete[n];
    for(int i=0; i<n; i++) isComplete[i] = 0;

    while (completed != n) {
        int index = -1, minBT = 1e9;

        for(int i=0; i<n; i++) {
            if (p[i].at <= time && !isComplete[i]) {
                if (p[i].bt < minBT) {
                    minBT = p[i].bt;
                    index = i;
                }
            }
        }
    }

```

```

        if (index != -1) {
            p[index].rt = time + p[index].bt;
            p[index].tot = p[index].rt - p[index].at;
            p[index].wt = p[index].tot - p[index].bt;

            sumAT += p[index].at;
            sumWT += p[index].wt;

            time = p[index].rt;
            isComplete[index] = 1;
            completed++;
        } else {
            time++;
        }

        printf("Average TAT: %.2f\n", float(sumAT)/n);
        printf("Average WT: %.2f\n", float(sumWT)/n);

        return 0;
    }

    output:
    Enter no. of processes: 2
    Arrival Time of P1: 0
    Burst Time of P1: 2
    Arrival Time of P2: 1
    Burst Time of P2: 2

    Average TAT: 3.50
    Average WT: 1.00

```

SJF pre-emptive

SFS
(P2)

Date: / /
Page: /

Algorithm: $1 \rightarrow$
struct process {

int id, at, bt, ct, tat, wt, rt, rem;
int maxn = 0;

int n;

printf("Enter no. of process: ");
scanf("%d", &n);

struct process p[n];
for(int i = 0; i < n; i++) {

p[i].id = i + 1;
printf("Arrival Time of P%d: ", i);
scanf("%d", &p[i].at);

printf("Burst Time of P%d: ", i);
scanf("%d", &p[i].bt);
p[i].rem = p[i].bt; p[i].wt = -1;

}
int completed = 0, time = 0, sumwt = 0;

while (completed != n) {

int idx = -1, minrem = 1e9;

for(int i = 0; i < n; i++) {
if (p[i].wt < time & p[i].rem < minrem) {

if (p[i].rem < minrem) {
minrem = p[i].rem;

idx = i;

}

}

if (idx == -1) {

if (p[idx].wt == -1) {

p[idx].wt = time - p[idx].ct;
sumwt += p[idx].wt;

if (p[idx].rem == 0) {

p[idx].ct = time;
p[idx].tat = p[idx].at + p[idx].wt;
p[idx].wt = p[idx].tat - p[idx].bt;

sumtat += p[idx].tat;
sumwt += p[idx].wt;

sumkt += p[idx].wt; completed++;
} else { time++; }

printf("Avg. TAT: %.2f", (float) sumtat/n);
printf("Avg. WT: %.2f", (float) sumwt/n);
printf("Avg. RT: %.2f", (float) sumkt/n);

return 0;

}

O/P:

Number of process: 3

Arrival Time of P1: 0 (0.0) 1.2

Burst Time of P1: 5 (5.0) 1.2

Arrival Time of P2: 1

Burst Time of P2: 3

Arrival Time of P3: 2

Burst Time of P3: 1

Avg TAT: 1.67

Avg WT: 1.67

}

Comparison:

PCRS:

P	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	7
P3	2	1	9	7	6

STF:
(NPE)

P	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	9	8	5
P3	2	1	8	9	3

STF:
(PE):

P	AT	BT	CT	TAT	WT
P1	0	5	9	9	4
P2	1	3	5	4	1
P3	2	1	3	1	0

Comparison:

Algorithm:

Avg WT Avg TAT

PCRS	3.33	6.33
STF (NPE)	2.67	5.67
STF (PE)	1.67	4.67

Date: / /
Page: /

(2)

Sum of left diagonal of matrix

Algorithm:

int main() {

int a[10][10], n; sum = 0;

printf("Enter order of matrix: ");

scanf("%d", &n);

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

scanf("%d", &a[i][j]);

if (i == j) sum += a[i][j];

printf("Sum of left diagonal: %d", sum);

return 0;

}

Enter order of matrix: 3

Elements: 1 3 5 6 7 8 9 2 4

Sum of left Diagonal: 12

←

Date: / /
Page: /

Sum of rows & column in matrix

```

3) #include <stdio.h>
int main() {
    int a[10][10], r, c, i, j, sum;
    printf("Enter no. of rows & columns: ");
    scanf("%d %d", &r, &c);

    for (int i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    for (i = 0; i < r; i++) { sum = 0;
        for (j = 0; j < c; j++) {
            sum += a[i][j];
        }
        printf("Sum of row %d = %d\n", i+1, sum);
    }

    for (j = 0; j < c; j++) { sum = 0;
        for (i = 0; i < r; i++) {
            sum += a[i][j];
        }
        printf("Sum of column %d = %d\n", j+1, sum);
    }
}

```

o/p: Enter no. of rows & columns: 2 2
Enter elements: 1 2 3 4

Sum of row 1 = 3
Sum of row 2 = 7
Sum of column 1 = 4
Sum of column 2 = 6

Count Duplicate elements in array

```

4) #include <stdio.h>
int main() {
    int a[100], n, i, j, count = 0;
    printf("Enter size: "); scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (j = 0; j < n; j++) {
        for (i = j+1; i < n; i++) {
            if (a[i] == a[j]) {
                count++;
                break;
            }
        }
    }

    printf("Number of Duplicates = %d", count);
}

```

o/p: Enter size: 10
1 2 2 3 4 5 4 7 8 8

Number of Duplicates: 2

② Delete element at desired position

```
#include <stdio.h>
int a[100], n, pos, i;

printf("Enter size: "); scanf("%d", &n);

for(i=0; i<n; i++)
    scanf("%d", &a[i]);

printf("Enter position to delete: ");
scanf("%d", &pos);

for(pos-1; i=n-1; i++)
    a[i] = a[i+1];

n--;

printf("Array after Deletion: ");
for(i=0; i<n; i++)
    printf("%d", a[i]);
```

o/p: Enter size: 6
1 2 3 4 5 6
Enter position to delete: 4
Array after Deletion: 1 2 3 5 6

Run
2/3/24

1 (c) Program to find TAT & WT Priority CPU scheduling algorithm.

Date: 10/3/24
Page: 5

```
#include <stdio.h>
struct Processes {
    int pid, at, bt, pr, ct, tat, wt, rt, fin;
};
```

```
void p1-non-preemptive(
    struct Processes p[10], int n) {
    int time = 0, completed = 0;
    while (completed < n) {
        int idx = -1, highest = 9999;
        for(int i=0; i<n; i++) {
            if (p[i].fin <= p[idx].at <= time) {
                if (p[i].pr <= highest) {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
        if (idx == -1) {
            time++;
            continue;
        }
        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        p[idx].fin = 1;
        completed++;
    }
```

```
printf("Non-preemptive Priority Scheduling");
printf("PID | AT | BT | PR | CT | TAT | WT | RT | FIN");
```

```

for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].rt,
        p[i].tot, p[i].wt);
}

```

```

void pri_preemptive(struct Process p[], int n) {
    int time = 0, completed = 0;
    while (completed < n) {
        int idx = -1, highest = 9999;
        for (int i=0; i<n; i++) {
            if (p[i].fin <= time) {
                if (p[i].pr < highest) {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
    }
}

```

```

if (idx == -1) {
    time++;
    continue;
}

```

```

p[idx].rt++;
time++;
if (p[idx].at == 0) {
    p[idx].rt = time;
    p[idx].tot = p[idx].rt - p[idx].at;
    p[idx].wt = p[idx].tot - p[idx].bt;
    p[idx].fin++;
    completed++;
}

```

```

printf("Preemptive Priority Scheduling");
printf("PID\tAT\tBT\tCT\tPR\tRT\tTOT\tWT");
for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].rt,
        p[i].tot, p[i].wt);
}

```

```

int main() {
    int n;
    printf("Enter no. of Processes: ");
    scanf("%d", &n);
}

```

```

struct Process p[n];
for (int i=0; i<n; i++) {
    p[i].pid = i+1;
    printf("Enter AT, BT & Priority for P%d, i);
    scanf("%d\t%d\t%d", &p[i].at, &p[i].bt, &p[i].pr);
    p[i].rt = p[i].bt;
    p[i].fin = 0;
}

```

```

struct Process p[n];
for (int i=0; i<n; i++) {
    p1[i] = p[i];
    p2[i] = p[i];
}

```

```

pri_non_preemptive(p1, n);
pri_preemptive(p2, n);

```

```

return 0;
}

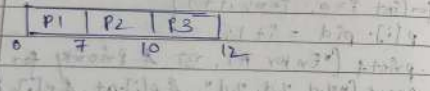
```


Enter no. of processes: 3
 Enter AT, BT, Priority for P1: 0, 7, 2
 Enter AT, BT, Priority for P2: 2, 3, 1
 Enter AT, BT, Priority for P3: 3, 2, 3

→ Non Pre-emptive Priority Scheduling:

PID	AT	BT	PR	CT	TAT	WT
1	0	7	2	7	7	0
2	2	3	1	10	8	5
3	3	2	3	12	9	7

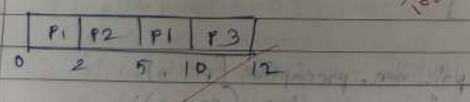
Gantt Chart:



→ Pre-emptive Priority Scheduling:

PID	AT	BT	PR	CT	TAT	WT
1	0	7	2	10	10	3
2	2	3	1	5	3	0
3	3	2	3	12	9	7

Gantt Chart:



Compare:

	Pre-emptive	non Pre-emptive
Avg-TAT	4.33	8.00
Avg-WT	3.33	4.00

+ Pre-emptive is better.

1(d) Round-Robin: Custom Size Quantum

```
#include <stdio.h>

struct Processes {
    int pid, at, bt, ct, tat, wt, rt, rem, start;
};

int main() {
    int n, tq;
    struct Processes p[20];

    printf("Enter no. of Processes: ");
    scanf("%d", &n);

    for(int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("Enter AT & BT for Process P%d: ", i+1);

        p[i].rem = p[i].bt;
        p[i].start = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int time = 0, completed = 0;

    int queue[100], front = 0, rear = 0;
    int visited[20] = 0;

    int gantt_pid[100], gantt_time[100];
    int g = 0, gantt_time[g] = 0;

    float sum_ct = 0, sum_tat = 0, sum_wt = 0;
    sum_rt = 0;
```



```

for (int i=0; i<n; i++) {
    if (p[i].at == 0) {
        queue | rear++ | j = i;
        visited[i] = 1;
    }
}

```

```

while (complete < n) {
    if (front == rear) {
        time++;
    }
    for (int i=0; i<n; i++) {
        if (p[i].wt <= time & !visited[i]) {
            queue | rear++ | j = i;
            visited[i] = 1;
        }
    }
    continue;
}

```

```

if (p[i].start == 0) {
    p[i].rt = time - p[i].at;
    total - rt += p[i].rt;
    p[i].start = 1;
}

```

```

if (p[i].start == 0) {
    p[i].rt = time - p[i].at;
    total - rt += p[i].rt;
    p[i].start = 1;
}

```

```

if (p[i].start == 0) {
    p[i].rt = time - p[i].at;
    total - rt += p[i].rt;
    p[i].start = 1;
}

```

```

if (p[i].start == 0) {
    p[i].rt = time - p[i].at;
    total - rt += p[i].rt;
    p[i].start = 1;
}

```

```

if (p[i].start == 0) {
    p[i].rt = time - p[i].at;
    total - rt += p[i].rt;
    p[i].start = 1;
}

```

```

p[i].rem = 0;
p[i].at = time;
p[i].tot = p[i].at - p[i].at;
p[i].wt = p[i].tot - p[i].bt;

```

```

sum - ct += p[i].at;
sum - tot += p[i].tot;
sum - rt += p[i].rt;
complete++;

```

```

for (int i=0; i<n; i++) {
    if (p[i].at <= time & !visited[i]) {
        queue | rear++ | j = i;
        visited[i] = 1;
    }
}

```

```

if (p[i].rem > 0) { queue | rear++ | j = i; }

```

```

p[i].rem = p[i].wt - p[i].bt;
p[i].at = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;

```

```

p[i].at = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;

```

```

p[i].at = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;

```

```

p[i].at = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;
p[i].wt = p[i].tot - p[i].bt;

```

```

for (int i=0; i<n; i++) {
    for (int k=0; k<n; k++) {
        p[i].wt = p[i].tot - p[i].bt;
    }
}

```


for (int i = 1; i <= j; i++)
 for (int k = 0; k < quantum[i-1]; k++)
 printf("%d %d", i, k);

Q/P:

Enter no. of processes: 5
 AT for P1: 0.5
 AT for P2: 1.3
 AT for P3: 2.1
 AT for P4: 3.2
 AT for P5: 4.3

Enter time quantum: 2

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	13	13	8	0
P2	1	3	12	11	8	1
P3	2	1	5	3	2	2
P4	3	2	9	6	4	4
P5	4	3	14	10	7	5

Gantt Chart:

P1 P2 P3 P1 P4 P5 P2 P1 P5
 0.2 4 5 7 9 11 12 13 14

Avg-TAT: 8.60

Avg-WT: 5.80

Avg-RT: 2.40

Enter time quantum: 3

PID	AT	BT	CT	TAT	WT	RT
1	0	5	11	11	8	0
2	1	3	7	7	2	2
3	2	1	4	5	4	4
4	3	2	9	6	4	4
5	4	3	14	10	7	5

Gantt Chart:

P1 P2 P3 P4 P1 P5
 0 3 6 9 11 14

Avg-TAT: 7.40

Avg-WT: 4.60

Avg-RT: 3.40

Conclusion: The Round Robin model with time quantum 3 is better compared to tq=2.

17/3/20

2. Multi-level queue scheduling w/ FCFS & SJF

```
#include <stdio.h>

struct Processes { int pid, at, bt, rt, tot, wt, rt, q; };

int main() {
    int n;
    struct Processes p[10];

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for(int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("Enter AT & BT for P%d", i+1);
    }

    void initQueue(queue *q) {
        q->front = q->rear = -1;
    }

    int compareArrival(const void *a, const void *b) {
        return ((Process *)a->at - ((Process *)b->at);
    }

    int main() {
        int n;
        printf("Enter no. of processes: ");
        scanf("%d", &n);

        Process process_list[10];
        for(int i=0; i<n; i++) {
            printf("Enter AT, BT and type: ");
            scanf("%d %d %s", &process[i].at, &process[i].bt, &process[i].type);
        }
    }
}
```

```
if (process[i].at < process[j].at)
    process[i] < process[j];
}

qsort(p, n, sizeof(Processes), compareArrival);

Queue sysQ, userQ;
initQueue(&sysQ); initQueue(&userQ);

int currentTime=0, completed=0, i=0;
Process *curr_process = NULL;
int time[10] = {0}, timeRemain = 0;

while (completed < n)
    while (i < n && p[i].at <= curr_time) {
        if (strcmp(p[i].type, "system") == 0)
            enqueue(&sysQ, p[i]);
        else
            enqueue(&userQ, p[i]);
        i++;
    }

    if (curr_process != NULL) {
        if (strcmp(curr_process->type, "system") == 0)
            if (!isEmpty(&sysQ))
                dequeue(&sysQ, &curr_process);
            curr_process = NULL;
        }
        else {
            if (curr_process == NULL) {
                if (!isEmpty(&userQ))
                    dequeue(&userQ, &curr_process);
                curr_process = P;
            }
        }
    }
}
```


3 Program to Simulate RTOs Algorithm
RHS 4 EDF: ~~hyper~~

#include <stdio.h>
#define max_time 100

struct Process {
int id, bt, per, dl, rem, next_arrival, na, abs_dl;
float wt, tat; } pro;

void check_schedulability(struct Processes p[3])

{
float u = 0;
for(int i = 0; i < N; i++)
u += (float) p[i].bt / p[i].period;

printf("CPU Utilization: %.3f", u);

float rm_bound = 1 + (2 * ((float) 1 / (n - 1)));
printf("RM Bound = %.3f", rm_bound);

if (u <= 1)
printf("EDF: Schedulable");
else printf("EDF: Non Schedulable");

if (u <= rm)
printf("RM: Schedulable");
else printf("RM: Non Schedulable");

void EDF(@pro p[3], int hyper) {
for(int i = 0; i < hyper; i++)

for(int i = 0; i < n; i++)

{
if (t == p[i].nextarrival) {
p[i].rem = p[i].bt;
p[i].abs_dl = t + p[i].dl;
p[i].nextarrival += p[i].period;

}
get idx = -1;
get min_dl = 100;

for(int i = 0; i < n; i++)
if (p[i].rem > 0 && p[i].abs_dl < min_dl) {
min_dl = p[i].abs_dl;
idx = i;

}
if (idx != -1) {
if (p[idx].rem == 0) {
p[idx].ct = t + p[idx].bt;
p[idx].tat = p[idx].ct -
(p[idx].na == p[idx].period);
p[idx].wt = p[idx].tat - p[idx].bt;

}
printf("ID BT DT CT WT Tat N");
for(int i = 0; i < n; i++)
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d",
p[i].id, p[i].bt, p[i].dl, p[i].ct,
p[i].wt, p[i].tat);


```

int main () {
    pro original[N] = {
        {1, 4, 4, 1, 0, 3},
        {2, 2, 5, 1, 0, 3},
        {3, 1, 0, 0, 0, 3}
    };
    //
    pro p[N];
    int hyper = find_hyper (original);
    printf ("hyperperiod (LCM) = %d\n", hyper);
    //
    // Check schedulability (original);
    printf ("EDF: ");
    EDF (p, hyper);
    //
    printf ("RM: ");
    RM (p, hyper);
}

```

n/p:

Hyper period (LCM) = 50

CPU Utilization = 0.700

RM Bound = 0.74801 (1/40 * 100)

EDF: Schedulable

RM: Schedulable

$u = \frac{1}{4} + \frac{2}{5} + \frac{1}{2} = 0.750$ Schedulable

$3(2^{1/3} - 1) = 0.780$ Schedulable

RHS	LEFT	Schedulling
-----	-----------------	-------------

ID	BT	PL	CT	WT	TAT
1	1	2	17	0	1
2	2	4	18	1	3
3	1	3	4	3	2

Handwritten number line from 0 to 19. Above the line, boxes contain the sequence: 1, 2, 2, 3, 1, 2, 2, 1, 2, 2, 3, 2, 1, 2. The box containing '3' at position 12 is highlighted with a red border.

EDF-Schätzung:

ID	GT	DL	CT	WT	TAT
1	1	2	17	0	1
2	2	4	18	1	3
3	1	3	2	1	2

1/p: P1: BT, DL, P = 2, 3, 3
P2: BT, DL, P = 4, 4, 4
P3: BT, DL, P = 3, 6, 6
P4: BT, DL, P = 2, 8, 8
CPU Utilization = 2.167
RM Bound = 0.450
EDF: Not Schedulable
RM: Not Schedulable

$$u = \frac{2}{3} + 1 + \frac{3}{6} = 2.16 \text{ not sel} \dots \because > 1$$

$$r_m = 2(2^4 - 1) = 0.75 \text{ not schedulable}$$

```

pthread_mutex_lock(&mutex);
sem_post(&full);
}

void* consumer(void* arg)
{
    int item;
    for(int i = 0; i < 15; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buf[out];
        out = (out + 1) % buf_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
}

int main()
{
    pthread_t p1, c1;
    sem_init(&empty, 0, buf_size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p1, NULL, producer, NULL);
    pthread_create(&c1, NULL, consumer, NULL);

    pthread_join(p1, NULL);
    pthread_join(c1, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Dining Philosophers Problem

All philosophers are thinking

P₀ picked left fork 0

P₀ picked right fork 1

P₀ is eating

P₁ picked left fork 2

P₁ picked right fork 3

P₂ is eating

P₁ puts down forks 2 & 3

P₁ is thinking

P₀ puts down forks 0 & 1

P₀ is thinking

P₀ picked up left fork 4

P₀ picked up right fork 5

P₀ is eating

#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define n 5

sem_t forks[n];

void* phil(void* num)

{
 int id = *(int*)num;

while(1){

printf("Philosopher %d is thinking\n", id);

sleep(1);


```

if (id % 2 == 0)
    sem_wait(&forks[id]);
printf("Philosopher picked left fork %d",
        id, id);
    sem_wait(&forks[id+1] % n);
printf("Philosopher %d picked right fork %d",
        id, id);
}
else {
    sem_wait(&forks[id+1] % n);
printf("Philosopher picked right fork %d",
        id, id+1 % n);
    sem_wait(&forks[id]);
printf("Philosopher %d picked left fork %d",
        id, id+1 % n);
}

int main() {
    pthread_t phil[n];
    int ids[n];
    for (int i = 0; i < n; i++)
        sem_init(&forks[i], 0, 1);
    for (int i = 0; i < n; i++)
        ids[i] = i;
    pthread_create(&phil[i], NULL, phil, &ids[i]);

    for (int i = 0; i < n; i++)
        pthread_join(phil[i], NULL);

    for (int i = 0; i < n; i++)
        sem_destroy(&forks[i]);
}

return 0;

```

For 28/10/20

Banker's Algorithm:

```

#include <stdio.h>
#include <stdlib.h>

bool isSafe(int n, int m, int alloc[n][m],
            int need[n][m], int avail[n], int safeSeq[]) {
    int work[m]; bool finish[n];

    for (int i = 0; i < n; i++)
        work[i] = avail[i];
    for (int i = 0; i < n; i++)
        finish[i] = false;

    int count = 0;
    while (count < n)
        bool found = false;
        for (int i = 0; i < n; i++)
            if (!finish[i])
                int j;
                for (j = 0; j < m; j++)
                    if (need[i][j] > work[j])
                        break;
                if (j == m)
                    for (int k = 0; k < m; k++)
                        work[k] += alloc[i][k];
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
            }
        }
    if (!found) return false;
    return true;
}

```

```

bool reqRes (int n, int m, int alloc[n][m],
int max[n][m], int need[n][m], int avail[m],
int process, int req[1]) {
    for (int i = 0; i < m; i++) {
        if (req[i] > need[process][i]) {
            printf("Error");
            return false;
        }
    }
    for (int i = 0; i < m; i++) {
        if (req[i] > avail[i]) {
            printf("Process must wait");
            return false;
        }
    }
    for (int i = 0; i < m; i++) {
        avail[i] -= req[i];
        alloc[process][i] += req[i];
        need[process][i] -= req[i];
    }
    int safeSeq[n];
    if (isSafe(n, m, alloc, need, avail, safeSeq)) {
        printf("Req granted. Safe state");
        printf("Safe Sequence: ");
        for (int i = 0; i < n; i++) {
            printf("P%d -> ", safeSeq[i]);
        }
        printf("P%d", safeSeq[n-1]);
        return true;
    } else {
        for (int i = 0; i < m; i++) {
            avail[i] += req[i];
            alloc[process][i] -= req[i];
            need[process][i] += req[i];
        }
        return false;
    }
}

```

```

int main() {
    int n, m;
    printf("Enter no. of processes: "); scanf("%d", &n);
    printf("Enter no. of resources: "); scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m], need[n][m];
    printf("Enter allocation matrix (n):");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
        printf("Enter max demand matrix (n):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                scanf("%d", &max[i][j]);
            }
        }
        printf("Enter available resources (m):");
        for (int i = 0; i < m; i++) {
            scanf("%d", &avail[i]);
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                need[i][j] = max[i][j] - alloc[i][j];
            }
        }
        int safeSeq[n];
        if (isSafe(n, m, alloc, avail, need, safeSeq)) {
            int process;
            printf("Enter process making request: ");
            scanf("%d", &process);
            int req[m];
            printf("Enter req vector: ");
            for (int i = 0; i < m; i++) {
                scanf("%d", &req[i]);
            }
            reqRes(n, m, alloc, max, need, avail, process, req);
            return 0;
        }
    }
}

```


1/p Enter no. of process: 5
 Enter no. of resources: 4
 Allocation matrix:
 2 0 0 1
 3 1 2 1
 2 1 0 3
 1 3 1 2
 1 4 3 2
 Max Demand Matrix:
 4 2 1 2
 5 2 5 2
 2 3 1 6
 1 4 2 4
 3 6 6 5
 Available Resources:
 3 3 2 1

sum is in safe state.

Safe Sequence: $P_0 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1 \rightarrow P_2$

Enter process making request: 1

Enter req vector: 1 1 0 0

Request can be granted. sum remains in safe state

Safe Sequence: $P_0 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1 \rightarrow P_2$

5b Deadlock Detection

```

#include <stdio.h>
#include <stdbool.h>

```

```

int main() {
    int n, m;
    printf("No. of processes: "); scanf("%d", &n);
    printf("No. of resources: "); scanf("%d", &m);

    int alloc[n][m], req[n][m], avail[m];
    bool fin[n];
    int deadlock[n], count = 0;

```

```

printf("Allocation Matrix:");

```

```

for (int i = 0; i < n; i++)

```

```

    fin[i] = false;

```

```

    for (int j = 0; j < m; j++)

```

```

        scanf("%d", &alloc[i][j]);

```

```

printf("Enter Request matrix:");

```

```

for (int i = 0; i < n; i++) {

```

```

    for (int j = 0; j < m; j++) {

```

```

        scanf("%d", &req[i][j]);

```

```

    }
}

```

```

for (int i = 0; i < n; i++)

```

```

    scanf("%d", &avail[j]);

```

```

for (int k = 0; k < n; k++) {

```

```

    for (int i = 0; i < n; i++) {

```

```

        if (!fin[i]) {

```

```

            bool canBeDone = true;

```

```

            for (int j = 0; j < m; j++) {

```

```

                if (req[i][j] > avail[j]) {

```

```

                    canBeDone = false;

```

```

                }
            }

```

```

if (not Done)
{
    for (int i=0; i<n; i++)
        avail[i] += alloc[i][j];
    free[i] = true;
}

for (int i=0; i<n; i++)
{
    if (b[i] <= p[i])
        deadlocked[i] = true;
}

if (count > 0)
{
    printf("System in DEADLOCK due to ");
    for (int i=0; i<n; i++)
        printf("%d", deadlocked[i]);
}
else
    printf("No deadlock");

return 0;
}

ip: No. of processes: 4
No. of resources: 3
Allocation:
10 2
2 1 1
1 0 3
1 2 2

Request:
0 0 1
1 0 2
0 0 0
3 3 0

Available:
0 0 0

System is in DEADLOCK due to P3:

```

First Fit, Best Fit & Worst Fit Algorithm

```

#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int n, m, type;
int a[100], b[100], p[100], free[100];
for (i=0; i<n; i++) a[i] = -1;
for (i=0; i<n; i++)
{
    p[i] = -1;
    for (j=0; j<m; j++)
    {
        if (b[j] >= p[i])
        {
            idx = j;
            if (type == 1 && b[j] < b[idx]) idx = j;
            if (type == 2 && b[j] > b[idx]) idx = j;
        }
    }
    if (idx != -1)
    {
        a[i] = idx;
        b[idx] -= p[i];
    }
    if (type == 0) printf("First Fit: %d\n", i);
    else if (type == 1) printf("Best Fit: %d\n", i);
    else if (type == 2) printf("Worst Fit: %d\n", i);
    for (i=0; i<n; i++)
    {
        printf("%d\t", p[i]);
        if (a[i] != -1) printf("%d\n", a[i]+1);
        else printf("Not Allocated\n");
    }
}

int main()
{
    int m, n, i;
    printf("Enter no. of blocks: "); scanf("%d", &m);
    int b[m], b2[m], b3[m];

```



```

printf("Enter block sizes\n");
for(i=0; i<ms; i++)
    scanf("%d", &b[i]);
b[i] = b[i] - b[i];
}
printf("Enter no. of processes\n");
scanf("%d", &n);
int p[n];
printf("Enter process sizes\n");
for(i=0; i<n; i++)
    scanf("%d", &p[i]);
allocate(b, n, p, 0);
allocate(b, n, p, 1);
allocate(b, n, p, 2);
return 0;
}

```

11. Enter no. of blocks: 5 10 1 1 0
 block sizes: 400 100 100 100 600
 Enter no. of processes: 4
 process sizes: 212 512 312 526 (312 = 526)
 First fit of 1st process: Best fit
 Process Block Process Block
 212 1 212 4
 512 2 512 5
 312 3 312 1
 526 not allocated

12. Worst fit:
 Process Block
 212 2
 512 5
 312 1
 526 not allocated

Page Replacement Algorithm using
 a) FIFO b) LRU c) Optimal

```

#include <stdio.h>

void fifo(int pages[], int n, int frames)
{
    int frames[frames], i, j, k=0, faults=0;
    for(i=0; i<frames; i++)
        frame[i] = -1;
    for(i=0; i<n; i++)
    {
        int found = 0;
        for(j=0; j<frames; j++)
            if (frame[j] == pages[i])
                found = 1;
        if (!found)
        {
            frame[k] = pages[i];
            k = (k+1) % frames;
            faults++;
        }
        printf("Page faults = %d", faults);
    }
}

```

```

void lru(int pages[], int n, int frames)
{
    int frames[frames], time[frames], i, j, pos, faults=0;
    for(i=0; i<frames; i++)
        frame[i] = -1;
        time[i] = -1;
    for(i=0; i<n; i++)
    {
        int found = 0;
        for(j=0; j<frames; j++)
            if (frames[j] == pages[i])
                found = 1;
        if (!found)
        {
            frame[pos] = pages[i];
            time[pos] = i;
            pos = (pos+1) % frames;
            faults++;
        }
    }
}

```

```

    found = 1;
    time[j] = i;
    break;
}
if (!found)
    pos = 0;
for (j = 1; j < frames; j++)
    if (time[j] < time[pos])
        pos = j;

frame[pos] = pages[i];
time[pos] = i;
faults++;
printf("LRU page faults: %d", faults);
}

```

```

void optimal(int pages[], int n, int frames)
{
    int frame[frames], i, j, k, pos;
    faults = 0;
    for (i = 0; i < frames; i++)
        frame[i] = -1;
    for (i = 0; i < n; i++)
    {
        int found = 0;
        for (j = 0; j < frames; j++)
            if (frame[j] == pages[i])
            {
                found = 1;
                break;
            }
        if (!found)
        {
            int farthest = -1;
            pos = 0;
            for (j = 0; j < frames; j++)
            {
                int next = -1;
                for (k = i+1; k < n; k++)
                    if (frame[k] == pages[j])
                        next = k;
                if (next == -1)
                    break;
            }
        }
    }
}

```

```

    if (next == -1)
        pos = j;
    break;
}
if (next > farthest)
    farthest = next;
pos = j;

frame[pos] = pages[i];
faults++;
printf("Optimal page faults: %d", faults);
}

int main()
{
    int n, frames, i;
    printf("Enter no. of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter ref string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter no. of frames: ");
    scanf("%d", &frames);

    fifo(pages, n, frames);
    lru(pages, n, frames);
    optimal(pages, n, frames);

    return 0;
}

```


Enter no. of pages: 10
 Enter ref string: 7 0 1 2 0 3 0 1 2 3
 Enter no. of frames: 3

~~FIFO:~~
 Page 7 → 7 -1 -1
 Page 0 → 7 0 -1
 Page 1 → 7 0 1
 Page 0 → 2 0 1 Hit
 Page 3 → 2 3 1
 Page 4 → 2 3 4
 Page 3 → 2 3 4

FIFO:
 Page 7 → 7 -1 -1
 Page 0 → 7 0 -1
 Page 1 → 7 0 1
 Page 2 → 2 0 1
 Page 0 → 2 0 1 Hit
 Page 3 → 2 3 1
 Page 4 → 2 3 4
 Page 2 → 2 3 4 Hit
 Page 3 → 2 3 4

~~FIFO page faults = 9~~

LRU:
 Page 7 → 7 -1 -1
 Page 0 → 7 0 -1
 Page 1 → 7 0 1
 Page 2 → 2 0 1
 Page 0 → 2 0 1 Hit
 Page 3 → 2 3 1
 Page 4 → 2 3 4 Hit
 Page 2 → 2 4 3
 Page 3 → 2 4 3
 Page faults = 8

Optimal:
 Page 7 → 7 -1 -1
 Page 0 → 7 0 -1
 Page 1 → 7 0 1
 Page 2 → 2 0 1
 Page 0 → 2 0 1 Hit
 Page 3 → 2 0 3
 Page 0 → 2 0 3 Hit
 Page 4 → 2 4 3
 Page 2 → 2 4 3 Hit
 Page 3 → 2 4 3 Hit

Optimal Page faults = 6

27/5/20

Practice: Consider 2 processes A & B. Two semaphores
 vars s & t are used to sync the processes.
 s initial 1 & t to 1. Processes are
 scheduled as: A B A B B A B A

```

Process A:
while (1) {
  wait(s);
  print("A");
  print("P");
  signal(t);
}

Process B:
while (1) {
  wait(t);
  print("B");
  print("I");
  signal(s);
}
  
```

s = 0, t = 1, remark

Process	s	t	State
A	0	1	A is blocked
B	1	0	B is blocked
A	0	1	11 PP
B	1	0	B is blocked
A	0	1	11 PP, 11 PP
B	1	0	11 PP, 11 PP
A	0	1	11 PP, 11 PP, 11 PP

Producer-Consumer problem using Semaphores

Operation: for each empty
 buffer size = 3
 in = 0, out = 0
 empty = 3, full = 0, mutex = 1

Operation	in	out	empty	full	mutex	State
Producer(A)	1	0	2	1	1	[A, -, -]
Producer(B)	2	0	1	2	1	[A, B, -]
Producer(C)	0	0	0	3	1	[A, B, C]
Producer	0	0	0	3	1	[A, B, C]
Consumer(A)	0	1	1	2	1	[-, B, C]
Consumer(B)	1	1	0	3	1	[0, B, C]
Consumer(C)	1	2	1	2	1	[0, -, C]

#include <stdio.h>

#include <semaphore.h>

int buf, buf_size, in = 0, out = 0;

sem_t empty, full;

pthread_mutex_t mutex;

void * producer(void * arg) {

int item;

for (int i = 0; i < 15; i++)

item = i;

sem_wait(&empty);

pthread_mutex_lock(&mutex);

buf[in] = item;

in = (in + 1) % buf_size;